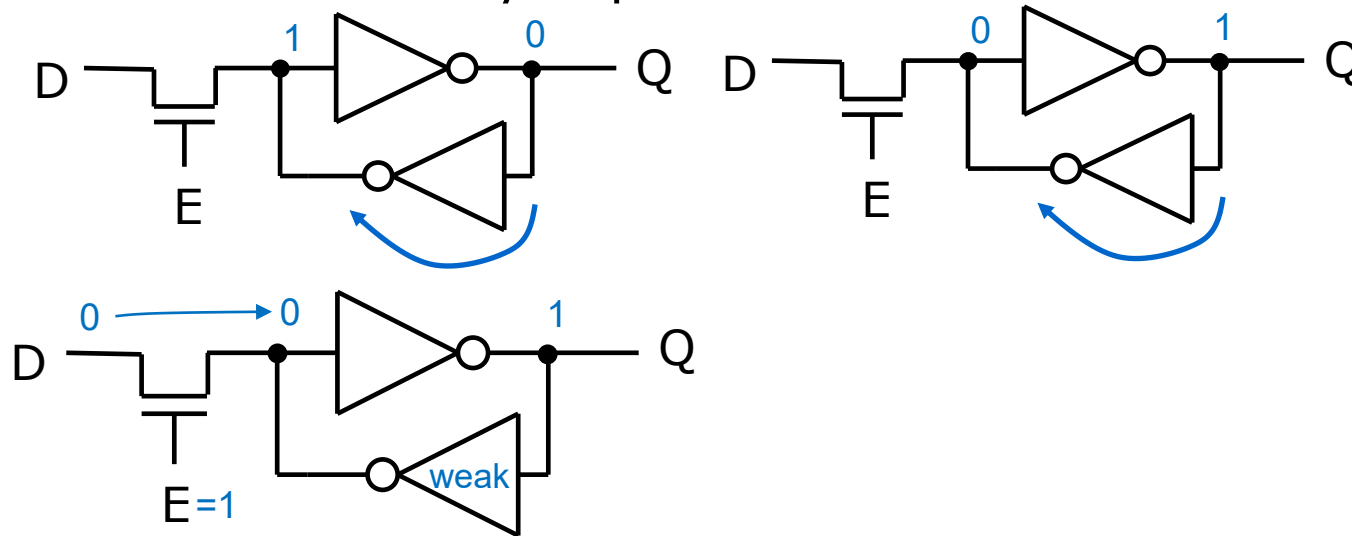


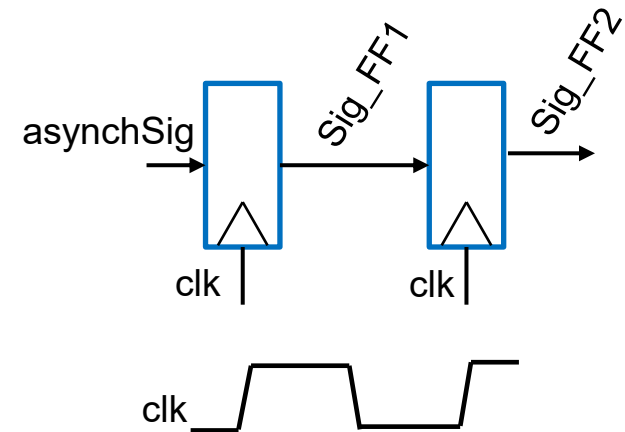
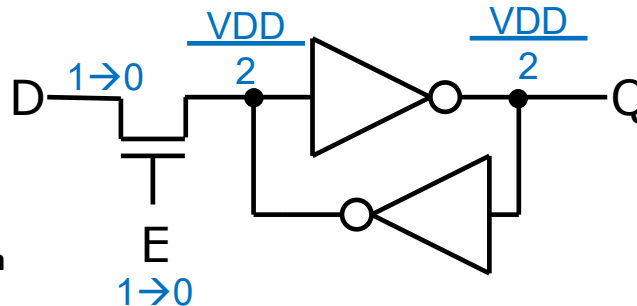
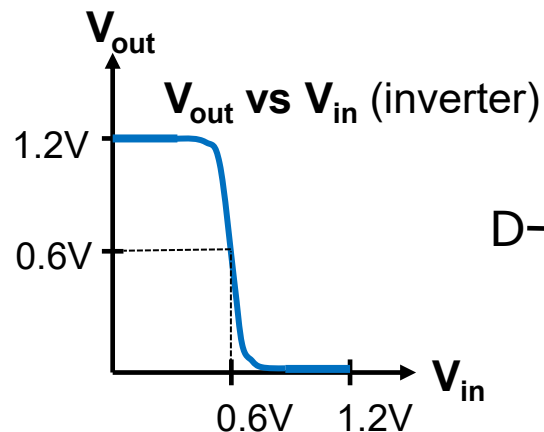


Metastability Recap

- Consider the memory loop in a latch



- May become metastable if signal is captured during transition



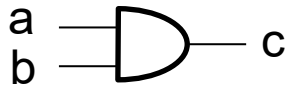


Dataflow Verilog Recap

- Continuous assign statements

logic a, b, c, d;

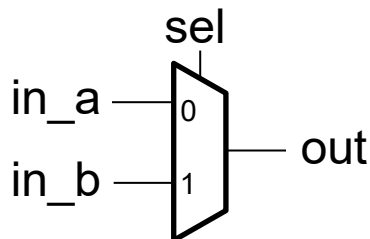
assign c = a & b;



assign d = ~a | b;

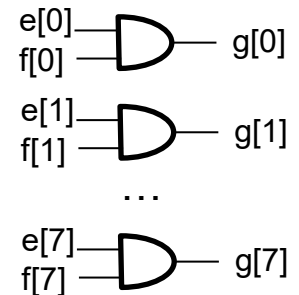
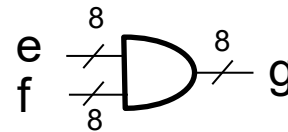


logic sel, in_a, in_b, out;
assign out = (sel)?in_b : in_a;

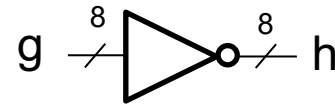


logic [7:0] e, f, g, h; // vectors

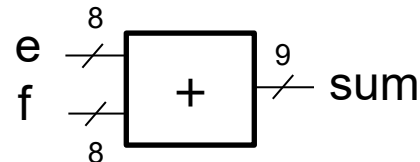
assign g = e & f;



assign h = ~g;



logic [8:0] sum;
assign sum = e + f;





Exercise 03

Part 1: Synchronizing Edge Detector

- Submit a **structural** Verilog file that models a module that can synchronize a signal to our system clock (**clk**) and output a signal that is high for 1 clock cycle when that signal has a rising edge. Assume a D-FF is already defined that has an interface of:(D, clk,Q,PRN) and a module name of **dff**.
 - What do you need to solve metastability?
 - What do you need to detect a rising edge?
- Name your file **synch_detect.sv** and ensure the module name is also **synch_detect**
- Your interface should have:
 - asynch_sig_in** → input to synchronize and detect rising edge on
 - clk** → clock signal
 - rst_n** → global asynch active low reset (initialize the circuit) (**preset** flops in this case)
 - rise_edge** → output of your block that is high for 1 clock cycle after rising edge occurs on



Exercise 03

Part 1: Synchronizing Edge Detector

- Download the provided **dff.sv** and **synch_detect_tb.sv**
- Create a Modelsim project and test your design. The test bench is self-checking and should give a happy message in the console window if your DUT is good.

→ Submit **synch_detect.sv**

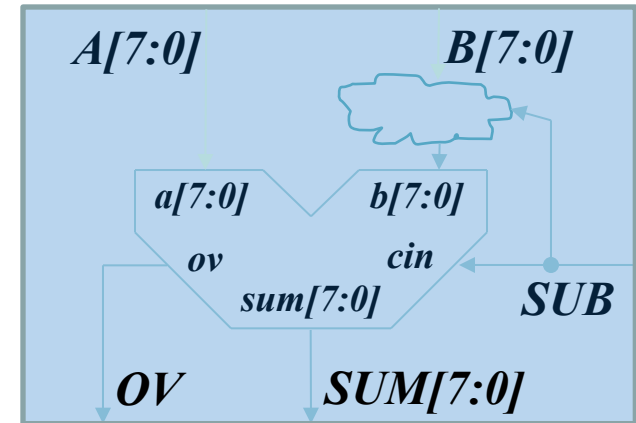


Exercise 03

Part 2: Testbench for Arithmetic Block

- Create a testbench for an 8-bit arithmetic block (provided as arith.sv)
- This arithmetic unit can perform addition or subtraction.

$$\begin{aligned} \text{SUM} &= A + B \text{ if SUB is low} \\ \text{SUM} &= A - B \text{ if SUB is high} \end{aligned}$$



arith.sv

- It also outputs an overflow signal if the result is either too positive or too negative to represent as an 8-bit **signed** number.
- The stimulus you apply should test both addition and subtraction.
- The stimulus you apply should test overflow conditions.
- arith contains a **bug** in the **ov logic**. See if your testbench can find it.
- Make sure you include comments describing your testcase.

→ **Submit arith_tb.sv and arith.sv** (fixing the bug in the **ov** logic)